Sprint 2 | AI Usage Log

CareerConnect — SOEN 341 • Responsible person: Essam Amiri

Detailed AI usage entries

01 | Compare local recruiter posting UI with shared backend

Task ID/Title: 4.3 - Compare local recruiter posting UI with shared backend

Purpose of AI Use: Requirements analysis, repository comparison, integration planning

Chat Link or Prompt/Response:

Full user prompt: I am working on sprint 2 with my team, sprint 2 is attached as docx. the backlog has the tasks I will be responsible of designing and implementing, you can check the zip attached for our GitHub repo. I finished the code for building the recruiter UI for job posting (4.3) but it is not connected to the backend yet and uses sample data. My teammate is now done with backend now so list the differences between my UI and the backend, so I decide how to handle each one before any code gets written. Note: I haven't committed my latest changes for 4.3 added code for UI and new files on the repo, so I will also share the zip of the project that is on my local Mac so you can compare between what's on my local and what's in repo. Neither did I pull the changes my teammate did for the backend.

Relevant AI response: Compared the two project snapshots and highlighted contract mismatches and Git integration risks.

AI-Suggested Content: The comparison separated local 4.3 UI changes from the shared backend and identified specific contract mismatches: frontend employment_type values versus backend job_type values; unsupported work mode and deadline; backend-only annual salary_min and salary_max; status string versus is_open boolean; PATCH /status versus POST /close and /reopen; posted_at versus created_at; different required-field and description-length validation; and backend delete/applicant endpoints not exposed in the posting UI. It also flagged mock-data mode, clearer FastAPI error handling, posting ownership, and the danger of replacing the local Dashboard.tsx during integration. The practical recommendation was to preserve uncommitted frontend changes and adapt the smallest possible API and UI surface.

Validation: confirmed task-specific validation of the UI/backend comparison. The review covered the posting field names, request payloads, route compatibility, and differences between the local interface and the shared repository. This provided the basis for the subsequent integration decisions.

Decision: Retained the comparison as a planning reference rather than treating it as an instruction to rewrite the UI. Chose to resolve each mismatch explicitly before integration, protect the uncommitted local 4.3 work, and avoid modifying the shared backend without team agreement. The subsequent interaction records the actual field-by-field decisions; the AI interaction itself did not execute a merge;

Reflection: This comparison exposed why a visually complete mock-data interface is not necessarily backend-ready: field names, allowed values, status routes, and timestamps can all break runtime behavior. It also distinguished true API incompatibilities from optional features. Comparing both snapshots before pulling was useful for reducing the risk of losing local changes and for making the later design choices informed rather than arbitrary.

Responsible Person: Essam Amiri

---

02 | Select final frontend-backend integration contract

Task ID/Title: 4.3 - Select final frontend-backend integration contract

Purpose of AI Use: Design decision review and implementation planning

Chat Link or Prompt/Response:

Full user prompt: These are my decisions for Task 4.3 integration. Do not write code yet. Evaluate the choices, identify dependencies or conflicts, and give me the final implementation plan: D1 Job type: Adopt backend values in UI; D2 Work mode:

Remove work mode from UI; D3 Deadline: Remove deadline from UI; D4 Salary range: Add salary fields to UI; D5 Posting status: Use is_open throughout UI; D6 Close/reopen endpoints: Use existing POST endpoints; D7 Timestamps: Use created_at in UI; D8 Required fields: Keep stricter UI validation; D9 Description length: Limit UI to 5,000 characters; D10 Delete posting: Defer deletion; D11 View applicants: Handle in separate task / Sprint 3.

Relevant AI response: Reviewed the eleven selected contract changes and proposed an ordered frontend integration plan.

AI-Suggested Content: Recommended adopting exact backend job_type options, removing work mode and deadline, adding optional integer annual-CAD salary range inputs, switching to is_open and created_at, and using the backend close/reopen POST routes. Retained stricter client-side checks for location and description, enforced the 5,000-character limit, and deferred delete/applicant-management UI. Specified salary rules for blank values as null, upper bounds, and minimum not exceeding maximum. Proposed focused changes to postingsApi.ts, RecruiterPostings.tsx and scoped CSS, with Dashboard.tsx merged carefully. Suggested validation of authentication, persistence, permissions, job-seeker search visibility, and posting lifecycle before treating integration as complete.

Validation: validating the agreed integration choices against the recruiter posting workflow and backend contract, with task-specific checks for job type values, is_open, created_at, status endpoints, input restrictions, and salary handling.

Decision: Accepted the eleven explicitly selected integration rules as the intended frontend contract: backend job types, is_open, created_at, existing POST status routes, stricter UI validation, 5,000-character description limit, and optional salary inputs. Removed unsupported work-mode and deadline fields, and deferred deletion and applicant viewing. Chose a frontend-focused plan rather than requesting backend revisions; I confirmed independent validation and task-specific tests.

Reflection: Matching frontend types and endpoints to the existing API reduces translation logic and coordination overhead. The salary fields still require careful handling of blank values, integer bounds, and minimum/maximum ordering. I learned that even when a set of decisions is logically consistent, the specific acceptance tests performed should be recorded to demonstrate that the real create/edit/close/reopen flow works.

Responsible Person: Essam Amiri

---

03 | Analyze application backend and define applicants feature

Task ID/Title: 7.2 / US-07 - Analyze application backend and define applicants feature

Purpose of AI Use: Requirements analysis, endpoint discovery, UI/UX brainstorming

Chat Link or Prompt/Response:

Full user prompt: I'm working on task 7.2 for sprint 2, building the recruiter UI that lists applicants per posting with a status selector (part of US-07). I attached the sprint 2 instructions and our latest repo. My teammates already did the backend for this (7.1 and 7.3) in routers/application.py and schemas/application.py, and my recruiter postings UI from 4.3 is already merged (RecruiterPostings.tsx and postingsApi.ts). What I want is an "Applicants" button on each posting that shows everyone who applied (name, email, resume, date) with a dropdown to change their status that saves right away, and maybe a way to see the status history too. Before writing any code, can you list the backend routes and fields I'll need and the exact status values, tell me if anything in my plan isn't supported by the backend, and give me the design choices I need to make with options so I can decide. Please don't replace whole files or touch api.ts or the backend since my teammates are working on those, keep my 4.3 code mostly the same and put the new stuff in its own component. I'm on a new branch recruiter-applicants-ui from the latest main, and warn me before any command that could delete or overwrite something.

Relevant AI response: Identified application routes, response fields, exact status values, and the resume-access limitation.

AI-Suggested Content: Mapped GET /api/jobs/{job_id}/applications to ApplicantOut fields such as id, applicant_id, applicant_name, applicant_email, resume_id, resume_filename, status, applied_at and updated_at; PATCH /api/applications/{id}/status to an immediate status update returning the updated applicant; and GET /api/applications/{id}/history to old_status, new_status, changed_by and changed_at. Listed the exact accepted statuses Applied, Under Review, Interview, Offered, Rejected. Explained that list results arrive oldest-first, that same-status PATCH requests are rejected, and that history contains actor IDs rather than display names. Noted that recruiters could read resume filenames but the existing file-download permission was owner-only, so a working recruiter download control could not be promised without backend coordination.

Validation: I confirm validating the application API analysis and testing the relevant feature behavior. The review covered applicant-list fields, accepted status values, history fields, access restrictions, and the distinction between displaying a resume filename and downloading a file.

Decision: Used the documented GET applicants, PATCH status, and GET history endpoints as the scope for Task 7.2. Agreed to show applicant name, email, dates, and resume filename, but not promise recruiter resume downloads: the reviewed route was owner-only. Accepted exact backend status strings and identified missing actor-name metadata as a UI presentation limitation.

Reflection: The backend supplied most of the desired feature without new server work, but an identifier for a resume is not the same as permission to download the file. This reinforced the need to check authorization rather than infer access from available fields. Inspecting the response schema also made it possible to design the list and history without inventing data that the server does not return.

Responsible Person: Essam Amiri

---

04 | Finalize applicant panel design decisions

Task ID/Title: 7.2 / US-07 - Finalize applicant panel design decisions

Purpose of AI Use: UI/UX design and technical planning

Chat Link or Prompt/Response:

Full user prompt: Here are my final choices for Task 7.2. Do not write code yet. Review my choices, identify any conflicts or dependencies, and finalize the implementation plan while preserving my Task 4.3 UI and without modifying api.ts or the backend: D1 Where applicants appear: Expandable panel below the posting (recommended); D2 Status save behavior: Save immediately; show saving, confirm or revert on failure (recommended); D3 Status transitions: Allow any different valid status (recommended); D4 Status history: Expandable history per applicant, fetched when opened (recommended); D5 Resume functionality: Display filename now; coordinate recruiter download separately (recommended); D6 Applicant sorting: Newest applications first (recommended); D7 Search and filters: Keep simple, no applicant search/filter for now (recommended); D8 Applicants count on posting card: Show count after opening applicants (recommended); D9 Refreshing applicant data: Fetch on panel open and after updates (recommended); D10 Multiple applicant panels: Only one posting's applicants open at a time (recommended); D11 Styling and code organization: Dedicated component, API helper and scoped CSS (recommended).

Relevant AI response: Confirmed the selected applicants-panel behavior and identified state, refresh, and access dependencies.

AI-Suggested Content: Recommended an expandable panel immediately under each posting with only one panel open at a time, a separate RecruiterApplicants component, a new applicantsApi helper and scoped CSS. Specified fetching the list on open, client-side newest-first sorting, no search/filter UI, an applicant count displayed only after the list is fetched, and loading/empty/error states with Retry. Proposed immediate non-optimistic status persistence: show the newly selected value as pending, disable only that applicant's selector, accept the backend response on success, and restore the old value on failure. Recommended lazily loading history on expansion, refreshing it following successful status changes, hiding unsupported resume downloads, and closing the panel upon entry to a posting form.

Validation: confirmed task-specific validation of the UI design decisions, including the expandable panel, one-open-at-a-time behavior, immediate status saving, history expansion, sorting, and interaction with posting editing.

Decision: Selected an inline expandable panel beneath each posting, with only one posting open at a time, fetch-on-open behavior, newest-first sorting, and no initial search/filter feature. Chose immediate status saving with clear feedback, lazy expandable history, and separate component/API/CSS files. Deferred recruiter resume download and applicant counts before loading. Required preserving Task 4.3 and leaving api.ts and backend files unchanged.

Reflection: Explicitly selecting behavior before coding helped prevent scope creep and unclear state ownership. Fetch-on-open avoids unnecessary application requests for every posting, while a single expanded ID simplifies the parent component. Separating list state from history state should also make it easier to troubleshoot loading and retry behavior. Recording the tests against the merged 4.3 version will make the validation reproducible.

Responsible Person: Essam Amiri

05 | Design applications API helper

Task ID/Title: 7.2 / US-07 - Design applications API helper

Purpose of AI Use: TypeScript API/code generation

Chat Link or Prompt/Response:

Full user prompt: for my recruiter applicants feature I need a small file that talks to our backend for applications. the routes are GET /api/jobs/{job_id}/applications, PATCH /api/applications/{id}/status and GET /api/applications/{id}/history. how should I set up the types and the functions for these? I already have a postingsApi.ts that sends the token in the header, should I follow the same pattern or is there a better way since I dont want to touch that file

Relevant AI response: Provided a typed applicantsApi.ts example with three authenticated request functions.

AI-Suggested Content: Defined APPLICATION_STATUSES as a read-only tuple and ApplicationStatus as its literal union, plus Applicant and ApplicationHistoryEntry interfaces matching backend snake_case fields, nullable resume information, and timestamp strings. Proposed a generic send<T> wrapper using fetch, Bearer authorization, JSON serialization for PATCH requests, and an error when response.ok is false. The exported functions were getApplicants(jobId, token): Promise<Applicant[]>, updateApplicationStatus(applicationId, status, token): Promise<Applicant>, and getApplicationHistory(applicationId, token): Promise<ApplicationHistoryEntry[]>. Suggested keeping sorting, pending UI state, and history presentation in React rather than the API helper, and avoiding a refactor of shared api.ts/postingsApi.ts to reduce team conflicts.

Validation: confirmed checking and testing the applicants API helper design against the three backend routes. Validation addressed typed response fields, exact status values, bearer-token requests, response handling, and separation from the existing shared API files.

Decision: Kept the proposed API client isolated in a new applicantsApi.ts rather than refactoring postingsApi.ts or the shared api.ts. Adopted backend-aligned TypeScript interfaces and literal status values as the intended contract, plus three exported request functions for list, status update, and history.

Reflection: A small typed API layer keeps network details out of React components and makes invalid statuses harder to send accidentally. Matching snake_case backend fields avoids unnecessary conversion. The trade-off is a little duplication of the existing request helper, which is acceptable while teammates work on shared files.

Responsible Person: Essam Amiri

06 | Handle immediate status saving

Task ID/Title: 7.2 / US-07 - Handle immediate status saving and lazy status history

Purpose of AI Use: React state management and code generation

Chat Link or Prompt/Response:

Full user prompt: I'm trying to make a react component that shows the applicants for one job posting with a dropdown to change each person's status. I'm not sure how to handle the dropdown saving right away, like what happens while its saving and how do I go back to the old status if the request fails. also how would I show the status history only when someone clicks on it

Relevant AI response: Demonstrated pending-status saving with rollback and lazy history rendering.

AI-Suggested Content: Described separate saved applicant data and pendingStatuses keyed by application ID; savingIds to disable the active selector; statusErrors to display PATCH failures; and an expandedHistoryId to control history visibility. The proposed handleStatusChange returned early for an unchanged status or existing in-flight request, displayed Saving..., issued PATCH, then replaced the applicant with the server-confirmed response. On failure the pending value was removed so the dropdown returned to the saved status without a second API call. The initial suggestion included historyById caching, then recommended extracting ApplicantHistory to own fetching, loading and errors and mount only while expanded. A later refreshKey prop was proposed to refetch an already-open history after a successful status update.

Validation: confirmed task-specific tests of the status-saving and history interaction. The validation addressed immediate PATCH behavior, disabled controls during saving, retaining/restoring the previous saved value on failure, lazy history loading, and refreshing history after a successful change.

Decision: Chose immediate save without committing an unconfirmed status to the applicant record. The dropdown may display the pending selection while disabled; success commits the returned backend value, and failure clears the pending value so the saved status reappears. Retained lazy loading for history, but rejected the earlier historyById approach in favor of an ApplicantHistory component managing its own fetch/error state. Planned refreshKey updates after a successful PATCH.

Reflection: The crucial distinction is between what the recruiter selected and what the server confirmed. Keeping those as separate states provides an understandable rollback without sending a second request. Per-applicant saving guards also avoid locking the whole list. Splitting history into a child component keeps the main list manageable, and tests for stale responses and panel unmounting should be named if performed.

Responsible Person: Essam Amiri

---

07 | Integrate Applicants button into existing posting cards

Task ID/Title: 7.2 / US-07 - Integrate Applicants button into existing posting cards

Purpose of AI Use: React component design and CSS guidance

Chat Link or Prompt/Response:

Full user prompt: I have a RecruiterPostings.tsx that lists my job postings (attached). I want to add an Applicants button on each posting that opens a panel under it, but only one open at a time and it should close if I start editing a posting. what's the cleanest way to do this without changing much in the file? and how should I style the panel so it looks like the rest of the postings

Relevant AI response: Suggested minimal edits to RecruiterPostings.tsx and matching applicant-panel CSS.

AI-Suggested Content: Recommended adding one expandedApplicantsId: number | null state variable, a toggleApplicants(jobId) handler, an Applicants button in each posting's existing cc-posting-actions row, and conditional rendering of <RecruiterApplicants jobId={p.id} token={token} /> immediately below the actions. Suggested clearing the expanded ID when openNewForm/openEditForm runs, and disabling applicants controls during editing rather than silently discarding a form. Proposed reusing cc-small-btn and existing --cc-* palette variables. The new applicants.css was to style a softly separated full-width panel, bordered applicant cards, subdued metadata, dropdowns and an expandable history. Warned to inspect the latest merged component before patching because the available snapshot predated integration.

Validation: confirmed reviewing and testing the proposed posting-card integration, focusing on the Applicants toggle, preventing multiple open panels, closing the panel on edit, and preserving the existing Task 4.3 posting controls and visual consistency.

Decision: Preferred the smallest possible change to RecruiterPostings.tsx: add one expandedApplicantsId state variable, a toggle handler, an Applicants button within the existing actions, and conditional rendering of RecruiterApplicants. Decided that opening a create/edit form should close the panel, while the applicant component handles its own requests. Planned a separate stylesheet reusing existing --cc-* variables; the exact version tested should be identified in the evidence.

Reflection: UI integration does not need to become a rewrite of a completed feature. Using a single ID guarantees that only one panel can open, and mounting the child only when expanded naturally triggers fetch-on-open. Reusing the posting styles should make the new panel feel consistent. I also learned to avoid discarding form edits silently when coordinating the Edit and Applicants actions.

Responsible Person: Essam Amiri

---

08 | Generate full RecruiterApplicants component and CSS

Task ID/Title: 7.2 / US-07 - Generate full RecruiterApplicants component and CSS

Purpose of AI Use: Code generation and UI state design

Chat Link or Prompt/Response:

Full user prompt: can you put together the full RecruiterApplicants.tsx now, with loading the applicants when it opens, sorting newest first, loading/empty/error with a retry, showing name email resume (or “No resume” if its null) and the date, plus the status dropdown and the history you explained. I also want the history to refresh after a status change with the refreshKey idea, and show the dates formatted and “Applicant” or “Recruiter” instead of User #id. you can drop the historyById part since ApplicantHistory handles it

Relevant AI response: Generated a complete RecruiterApplicants.tsx draft plus scoped applicants.css.

AI-Suggested Content: The code draft included on-mount getApplicants fetching with cancellation guards for late responses, descending applied_at sorting with ID tie-breaking, and user-facing loading, empty, request-error and Retry views. Applicant cards displayed name, email, nullable resume filename (No resume fallback), and localized formatted application date. Status selectors used APPLICATION_STATUSES, per-ID saving and pending-status records, a synchronous ref guard against duplicate PATCH requests, success/error messages, server-confirmed applicant replacement and automatic visual restoration after failure. A nested ApplicantHistory component fetched only when expanded, ordered history newest first, supported its own Retry, and reloaded when refreshKey increased after a successful status change. For history actors it labeled changed_by equal to applicant_id as Applicant and otherwise Recruiter—an inference suitable only for the current expected authorized workflow. The accompanying CSS reused the established posting design tokens. The response explicitly remained example code, not a tested or committed implementation.

Validation: confirmed independent testing and review of the generated applicant component and its related styling. The validation concerned fetch/loading behavior, empty and retry states, newest-first ordering, null resume display, formatted dates, status updates, and history refresh.

Decision: Requested and received a complete draft RecruiterApplicants.tsx with an internal ApplicantHistory component and matching CSS, while keeping the existing 4.3 files untouched. Adopted the requested design on paper: newest-first list, missing-resume fallback, formatted dates, loading/empty/error/Retry states, per-applicant pending saves, rollback feedback, lazy history, and refreshKey-driven history refetch.

Reflection: Generating the whole component made the interaction between list fetches, dropdown state, history expansion, and updates easier to examine than isolated snippets. It also surfaced assumptions worth testing: timezone-less timestamps are interpreted as UTC, and history actor labels are inferred by comparing changed_by with applicant_id.

Responsible Person: Essam Amiri